

TESTING: no asegura la calidad. Aseguramiento de calidad (QA) de testing no es lo mismo. Tiene como propósito enumerar los defectos y se asume que tiene. El desafío es encontrar esos defectos. Una vez terminado el testing no se puede asegurar, garantizar la calidad, ni asegurar que no hay mas defectos. La idea es evidenciar los defectos encontrados.

Aseguramiento de calidad: se da durante el proceso de construcción del software. La calidad es una decisión y eso determina el conjunto de acciones insertadas en el proceso mientras el producto se esta construyendo.

Calidad: es una cuestión subjetiva para cada individuo. Es una percepción personal. Adecuación de un producto o servicio a las necesidades y expectativas del que lo pidió. Tiene que ver con el nivel de cumplimiento del producto/servicio a las necesidades y expectativas. Se separan los términos necesidades y expectativas, ya que las expectativas no son manifestadas (son cosas que quiere y nunca lo dice el cliente, se relacionan con los RNF, “era obvio que debía hacer tal cosa...”).

El testing es un proceso de control y corroboración contra algo. Una de las cosas más difíciles es definir que se quiere construir, compuesto por elementos manifiestos y no manifiestos. No tiene posibilidad de garantizar calidad y que el producto no tiene defecto; lo único que puede hacer es hacer visibles algunos defectos que tenga el software. Es parte de lo que se hace para asegurar calidad.

El aseguramiento de calidad (control de calidad de proceso, producto y prueba de software) tiene como propósito acciones que acompañan el desarrollo del producto de software para que llegue a un testing exitoso (se encuentran defectos, los errores están implícitos en cualquier software) o no exitoso (no se encuentran defectos). La administración de configuración nos da las necesidades básicas de repositorio, ambiente y producto para que se puedan llevar a cabo las actividades de testing y QA.

Validación: lo hace el cliente porque es el único que puede determinar si lo que se ha hecho es correcto.

Verificación: determina si el producto funciona correctamente.

MVP valida si el producto es correcto, VALIDAR siempre va en un contraste con la realidad.

Error y Defecto NO son sinónimos. El error se da cuando queda en el contexto de la misma fase donde apareció, un error de requerimientos que se detecta en actividades de verificación y validación es un error (es lo deseable y es mas barato que corregirlo en fases posteriores, lo que aumenta el costo). Si un error se encuentra en etapas posteriores a la de requerimientos, se trata de un defecto.

Una falla es un mal funcionamiento del sistema que ocurre como consecuencia de un defecto; no todos los defectos provocan fallas, como problemas de diseño de interfaz, colores, estética.

Testing se ocupa de defectos. Testing no escribe código, a lo sumo escribe las pruebas. Pero lo que testing prueba es lo construido durante el análisis, diseño e implementación.

El testing es un proceso destructivo de tratar encontrar defectos (cuya presencia se asume) en el código. Se debe ir con actitudes negativas para demostrar que algo es incorrecto. El testing correcto es el que encuentra defectos. El 30 o 50% del costo de un software confiable. Un proceso de testing completo incluye pruebas estáticas y dinámicas con el propósito de identificar defectos. La calidad no se puede inyectar al final.

Siempre es mas barato hacer QA para la detección temprana de defectos.

¿Todos los defectos son iguales? NO. Aparece la necesidad de la clasificación de defectos para organizar la corrección.

Severidad: cuan grave es el defecto. Cuanto afecta al producto el defecto. El cliente no puede determinar la severidad, sino que lo hace el equipo de testing. Daño que le causa al software un defecto.

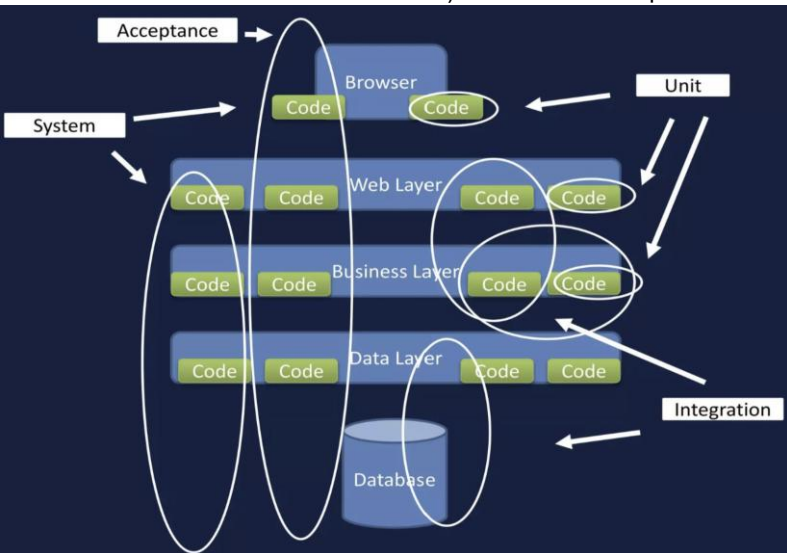
1. Bloqueante: no permite la utilización del sistema.
2. Crítico: no puedo cumplir el objetivo con la funcionalidad y no hay alternativas.
3. Mayor: no puedo cumplir el objetivo con la funcionalidad y hay alternativas.
4. Menor: no impide el cumplimiento de la funcionalidad, a lo sumo tiene advertencias. Por ejemplo: un error en un filtro.
5. Cosmético: errores de diseño, usabilidad en términos de disposición de menú, etc.

Prioridad: tiene que ver con la urgencia del negocio. El PO/Usuario final define la prioridad del defecto.

1. Urgencia: no se puede esperar a resolver el defecto. Manifestación de deseo.
2. Alta
3. Media
4. Baja

Niveles de prueba:

1. Pruebas Unitarias: se debe hacer por un desarrollador. Integradas en la TDD Test-Driven-Development (método de desarrollo de software). Se suele llamar primer nivel de testing. Son la excepción de la regla a nadie debe probar su propio código. Se hacen en el workflow de la implementación.



propio código. Se hacen en el workflow de la implementación.

2. Pruebas de integración/interfases: el foco de la prueba es probar que los componentes funcionen bien en conjunto (interfaces previstas y requeridas). El resultado de la prueba es un Build. Se hacen en el ámbito de la implementación o si se trabaja con “integración continua” se hace cuando se hace un pull request en un servidor de pruebas.

3. Pruebas del sistema: son propiedad del workflow de pruebas. Se las conoce como pruebas de versión también. La debe hacer alguien independiente a quien construyó.

4. Pruebas de aceptación de usuario (UAT): de estas pruebas depende el cumplimiento del contrato. Se hacen en el despliegue.

Ambientes o Environments: configuración del espacio de trabajo que utilizaremos en cada momento. El ambiente de testing y de desarrollo son diferentes. Implica las herramientas y todo lo necesario para trabajar, debe estar planificado en el plan de gestión de configuración (es donde se especifican ambientes y herramientas en cada ambiente). Es importante separarlos, ya que necesitamos resolver el problema de “en mi máquina andaba”. Además la objetividad de los que hacen testing, donde los desarrolladores no tengan inferencia. El ambiente de despliegue aéreo tiene dos: el de Pre-producción y de Producción. El de pre-producción es donde se realizan pruebas de aceptación. Este ambiente es caro y no puede garantizar al 100% que es igual al de producción. Hay alternativas a los ambientes de pre-producción con el despliegue y hay estrategias de despliegue, vinculadas con la entrega continua y el deployment continuo que buscan resolver la problemática. Lamentablemente, las pruebas que se hacen en el ambiente de prueba son únicamente funcionales.

CASO DE PRUEBA: set de condiciones o variables bajo las cuales un Rearte determina si el software está funcionando. Paso a paso que le permiten a otra persona llegar a un objetivo. Es el entregable más importante del testing. Es lo que nos permite hacer testing sistemático y metodológico. Un defecto que no se puede reproducir, no es un defecto. Para una buena definición de casos de prueba nos ayuda a reproducir defectos.

El título del caso de prueba es una prueba de aceptación. Es la unidad de la actividad de la prueba. Consta de tres partes:

- Objetivo: la característica del sistema a comprobar
- Datos de entrada y de ambiente: datos a introducir al aparta que se encuentra en condiciones preestablecidas.
- Comportamiento esperado: salida o acción esperada en el aparta de acuerdo a los requerimientos del mismo.

Pruebas exhaustivas: nunca se pueden probar todas las combinaciones posibles.

Para diseñar los casos de prueba sacamos las pruebas de los requerimientos en forma de las user stories. Debemos ser capaces de realizar verificación y validación.

El proceso estándar para testing tiene:

1. Planificación de pruebas: su artefacto es el plan de pruebas (estrategia, ambientes, e información de casos de prueba, métodos)
2. Diseño de CP: su artefacto son los casos de prueba y los datos de prueba

3. Ejecución: su artefacto es la versión de software
4. Reporte de defectos: su artefacto es el reporte

Se hace un ciclo entre la ejecución y reporte.

Estrategias:

Cada Negra, Caja Blanca

Regresión: al concluir el choclo de pruebas y reemplazarse la versión del sistema sometido al mismo, debe realizarse una verificación total de la nueva versión, a fin de prevenir la introducción de nuevos defectos al intentar solucionar los detectados. “Cada defecto que se arregla produce tres”, Las pruebas sin regresión generan defectos ocultos (sin embargo es la que mas se usa).

